

CarRacing PPO in Practice • Part 4

Collapses, Gremlins, and a 34-Step Debugging March

A hand-written PPO training log • From the illusion of eval 452 to iter60's perfect racing line • With a full post-mortem of every failure

Contents

- **0.** Background: where we stand
- **1.** The first false victory: eval 452 was scraped together by spinning
- **2.** Reward-misspec boomerang: an off-track penalty of -1 trained the car to "play dead"
- **3.** KL explosion & critic decoupling: the full mechanics of post-peak collapse
- **4.** The std starting point: std=1 is the root of all the wobble
- **5.** Conquering the S-curve: from grazing grass to the final touch of exit-recentering
- **6.** Four checkpoint traps: resume / opt / eval-across-steps / a fake KL guard
- **6.5** More lethal traps: warmup freeze / resume-nudge / eval_every / perturbation / truncation bias
- **7.** The 2000-step gremlin: how a loose horizon feeds reward hacking
- **7.5** A "zoo of attractors": spinning / playing dead / grass-edge crawling / flickering / circling / reversing...
- **8.** Two late insights: smart sampling & separating training-cost from eval-reward
- **9.** Methodology distilled: read numbers + gif, one variable at a time, checkpoint every step
- **10.** Final result: the full path from collapse to iter60's "perfect line" (with gif / model links)

0. Background: where we stand

In Parts 1–3 we hand-wrote PPO's three cores from scratch — a continuous Gaussian policy, GAE, and the clip surrogate loss — and got it training on CarRacing-v3. Part 4 records the hardest and most valuable stretch: **debugging an agent that "runs but crashes" into one that drives a perfect racing line, one step at a time.**

This stretch is almost all failure. But it is exactly these failures that finally taught me why RL is fragile, where the "gremlins" come from, and the gulf between "the number looks good" and "the

policy is actually good." So this write-up records the **phenomenon, diagnosis, root cause, and lesson** of every failure — because anyone doing CarRacing hits nearly all of these.

Core setup (used throughout)

Network: Nature-DQN CNN (Conv $8 \times 4 \rightarrow 4 \times 2 \rightarrow 3 \times 1 \rightarrow \text{FC512}$) + continuous Gaussian policy head + value head.

Algorithm: PPO, epochs=1, mb=1024, clip=0.2, GAE($\gamma=0.99$, $\lambda=0.95$), KL early-stop.

Environment: CarRacing-v3, 4-frame stack, raw reward (new tile +3.4 / per-frame -0.1 / off-field -100).

Evaluation: 5 fixed-seed tracks $\times$ deterministic action (Gaussian mean) $\times$ averaged.

1. The first false victory: eval 452 was scraped together by spinning

Phenomenon: in an early run, eval climbed from -101 all the way to **447**, looking about to be solved (CarRacing max ≈ 900 , official solved line 900). I was thrilled.

Collapse: after peaking at 447 at it100, it dropped off a cliff to -79 between it110-120, then stayed stuck at -82 forever.

Even weirder: on the collapsing iteration, kl / clipfrac / explained variance were all normal — by every available metric, that "fatal update" looked harmless.

Diagnosis: read the gif frame-by-frame, don't stare at numbers

The numbers found no cause, so I tore apart the eval gifs of it110 and it120 frame by frame. The truth was jarring: **the car was spinning in place on the grass, drawing spiral tracks behind it.**

Going back to check the environment's reward mechanics revealed the key facts:

- Reward comes from **visiting new tiles** (+3.4 each = $1000/N$), not from "being on track."
- Reward $\propto$ speed (faster $\rightarrow$ more tiles eaten per unit time).
- High speed $\rightarrow$ can't corner $\rightarrow$ flies off the track.

Root cause: eval 447 wasn't "driving," it was "banking tile score on a high-speed straight before flying off." After spinning off it earns nothing, but the score already banked stays. **The number 452/447 is the product of "brute-force flooring it" — it learned only to accelerate, and the moment it hit a limit (high-speed cornering) it collapsed, and couldn't come back (the critic collapsed too).**

Lesson 1: high eval $\neq$ good policy. A single peak with high variance (± 143) is very likely "luck + a shortcut." From here on I held to one rule: read numbers for trend and variance, read the gif for actual behavior, and when they disagree, trust the gif.

2. Reward-misspec boomerang: an off-track penalty of -1 trained the car to "play dead"

Since the collapse was "flying off track into grass," the instinct was to **add an off-track penalty so it wouldn't leave the track**. So I added a -1 /frame penalty for "all wheels on grass."

Catastrophic result: the agent learned to **stay perfectly still (play dead)**. eval -1045 .

Mechanism: an off-track -1 was too heavy $\rightarrow$ "don't move / stay put" became the safest policy (no off-track = no -1) $\rightarrow$ std stayed frozen at 1.0 , the policy locked into the "playing dead" absorbing state.

Dropping the penalty from -1 to -0.3 eased the playing-dead problem, but a new one appeared:

ev rises but eval is stuck: the critic's explained variance climbed (value is learning), but eval stayed pinned at -82 , and std stayed 1.0 (the policy wasn't learning at all).

Mechanism — "the big number swallows the small": the off-track penalty blew up the return variance (on-track episode ≈ -50 , grass episode ≈ -800). The magnitude of value loss (100) dwarfed policy loss (1) $\rightarrow$ the policy's gradient signal was drowned $\rightarrow$ std didn't drop, the policy couldn't learn.

Diagnosis: add the penalty and watch it collapse, delete it and watch it live

I ran a controlled comparison (one variable): remove the off-track penalty entirely, back to raw reward. Result:

Version	Pre-train eval	Phenomenon
off-track penalty -1	-780	plays dead, std locked at 1.0
off-track penalty -0.3	-275	ev rises but eval stuck, std won't drop
raw reward (penalty removed)	$+56$	positive feedback exists, std starts dropping, policy actually learns

Root cause: the off-track penalty created a local optimum where "not moving = safe." Under raw reward, grass itself is only -0.1 /frame (same as an already-visited tile on track), so

"moving toward the track" always has positive gradient — **the policy has a direction to learn, so std can drop.**

Lesson 2: reward is the most sensitive thing — don't touch it if you can avoid it. The agent will faithfully maximize whatever reward you write — write "penalize going off-track" and it learns "never move at all." The cost of a misspecified reward is catastrophic and counter-intuitive. This trap kept warning me later: prefer stabilization that doesn't touch reward (clip, mb, lr, std init) over adding penalty terms.

3. KL explosion & critic decoupling: the full mechanics of post-peak collapse

Back on raw reward, the policy started really learning. eval oscillated upward, peaking at 400–700 many times. But **almost every peak was followed by a collapse**. It took me a long time to piece together the full mechanics — and it turned out there were two different versions.

Version A: KL explosion (epoch reuse + high variance)

Early run (epochs=4, n_ep=2): after peaking at 447, it107 kl=0.105, it109 kl=0.149, it119 kl=0.21 — **consecutive KL explosions**. The policy was pushed away, the critic couldn't keep up, eval 447 → 233 → -79, dead.

Root cause: reusing one batch 4 times (epochs=4) + high-variance advantage from n_ep=2 → updates accumulated in the same direction → the KL snowballed.

Fix: drop epochs to 1 (no reuse), raise mb to 1024, raise n_steps from 2000 to 20000 (n_ep from 2 to 20, variance plummets). This combination suppressed the KL explosion — but planted the later seed of "mb too large so std won't drop."

Version B: critic-decoupling collapse (peak → return scale blows up → ev goes negative)

Later run: after eval peaked at 676/702, a new kind of collapse appeared.

- vL (value loss) shot from 5 to **99**;
- ev fell from 0.9 to **negative (-0.06, even -1.5)**;
- eval collapsed to -93, dead.

Root cause — the critic "chasing a big return alone":

1. peaking (e.g. 702) → the return scale suddenly grows;
2. if at this moment the KL guard **freezes the policy completely** (kl=0, no update) — but the KL guard only governs the policy, not the critic;

3. the critic keeps updating alone, trying to match the grown return, but the data it has was sampled by the old policy → **its target and its data are decoupled** → the critic chases itself into chaos → ev goes negative → the advantage is all wrong → the policy is dragged down by the wrong signal.

Lesson 3: too strict a KL guard backfires. Freezing the policy ($kl=0$) isn't safety — it decouples the critic from the policy. A trust-region mechanism must be "just tight enough" — too loose can't stop explosions, too tight manufactures decoupling collapse. This balance is very hard to find (thresholds $4\times/8\times/20\times$ were all tried, each with its own way of collapsing).

4. The std starting point: std=1 is the root of all the wobble

Even with KL suppressed, one stubborn problem remained: **the deterministic policy (eval) always "wobbles"** — after clearing the first corner, the steering swings left and right, wider and wider, until it swings into the grass.

Diagnosis chain: wobble → recentering → std too large

Frame-by-frame, the wobble's essence was "**steering that doesn't recenter**": enter a corner and steer → on exit you should recenter to go straight, but recentering fails → the car keeps drifting → over-corrects the other way → over-shoots → oscillation diverges.

Digging deeper: why does recentering fail? Literature calls this by name — **steering oscillation / action jitter** — and one root cause is exploration noise being too large. My `log_std` was initialized to `zeros`, i.e. **std = $e^0 = 1.0$** .

Why std=1 is fatal: steering output = mean $\pm$ a wide random range (± 1). **Even if the mean is correct (should recenter → output 0), the large noise of std=1 yanks the action to ± 1** , and recentering is ruined by the noise. No matter how accurate the mean, it doesn't help.

Fix: `log_std_init = -1` (std ≈ 0.37)

Following SB3's tuned CarRacing config (`log_std_init=-2`), I lowered the initial std from 1.0 to ≈ 0.37 :

```
self.log_std = nn.Parameter(torch.ones(act_dim) * -1.0) # std $\approx$ 0.37
# with a lower clamp(min=-2) to prevent std collapsing to 0
```

Immediate effect: switching to std=0.37, on the same training:

- eval variance dropped from **± 138 to $\pm 2-4$** (tracks became consistent);
- actions became precise, steering no longer swung wildly;

- at 200k steps eval hit 452 and didn't collapse (previously it took 100 iters and collapsed right there).

Lesson 4: initialization decides whether optimization can reach a good solution. The std starting point (1.0) is bad both ways: drop too fast → premature convergence into a fragile "one-trick pony" (run02); don't drop → the mean never sharpens, and it wobbles. Changing the starting point (0.37) sidesteps the dangerous "drop from big to small" process entirely — from the start it learns precise control under small noise. log_std_init is a badly underrated hyperparameter.

One level deeper: I later realized std=1's large noise also frequently yanks steering to the "hard-lock" extreme → triggers spinning in place → falls into the "spin ever further off" absorbing state. **In tasks with "dangerous extreme actions," a large std actively pushes the agent into an absorbing state.** A small std keeps actions steady near the mean, avoiding the dangerous extreme.

5. Conquering the S-curve: from grazing grass to the final touch of exit-recentering

After std=0.37, the policy began methodically conquering corners one by one. This is the stretch that felt most like "watching an agent grow," and its full arc of conquering the S-curve is worth recording (the S-curve is the hardest point on this track — consecutive reversing corners requiring "left-right-left" continuous counter-steering + full-time speed control).

Stage	Behavior (frame-by-frame)	Interpretation
early	plows straight into grass, stuck without exit	S-curve not learned at all
middle	the point it flies off the grass shifts back "half a small bend" each time	learning the S-curve segment by segment (a soft piecewise function fitting itself)
breakthrough	skims the S-curve edge at high speed (rapid road/grass flicker)	can pass, but by "grazing," not cleanly
conquered	passes S-curve + third corner in a row, eval jumps 452 → 676	mastered the core "continuous counter-steer" skill

Key insight: entry was learned, exit-recentering wasn't

In the "can pass but not stable" stage, frame-by-frame revealed a very fine phenomenon — the problem is **not entry but exit-recentering**:

- one gif: after exiting the S-curve, **over-recentered** → spun the other way;

- another: after exiting, **under-recentered** → kept circling the same way.

Both are "exit steering-recentering isn't controlled" — over / under. And it lines up with the numbers: large eval variance (± 237) and repeated KL spikes are exactly the policy making large repeated adjustments to the "recentering angle."

Lesson 5: ability grows skill by skill, and can be decomposed to a very fine grain (entry vs. exit-recentering). Understanding which skill, and which part of it, the agent is stuck on is far deeper than reading the eval number. This ability — "reading what the policy is learning from the gif" — is the most valuable takeaway of the whole project.

On the ep_ret vs eval reversal: during the conquering phase, ep_ret (exploratory) sometimes ran far above eval (deterministic). Key: **eval measures 5 fixed tracks, ep_ret measures 20 random tracks** — they run entirely different tracks and cannot be directly compared as "exploration vs deterministic." This confusion tripped me once: I briefly used it to wrongly argue "exploration potential," only later realizing it was a track-difficulty difference.

6. Four checkpoint traps: how engineering details waste a whole night

After conquering the S-curve and reaching eval 676/702, I wanted to "continue training from this good policy," so I started wrestling with checkpoint recovery. This stretch is almost all engineering traps — each can waste hours of training. They aren't algorithmic, but they are just as lethal.

Trap 1: resume=False, "continue training" becomes "train from scratch"

In code, `train()` defaulted to `resume=False`, and the call was also `train()`. The cell copied `best.pt` in, but train never loaded it — **it initialized a fresh network from zero**. The copied 676 was for nothing.

Fix: `train(resume=True)`. This trap was caught in code review — not reviewing means a wasted night.

Trap 2: best.pt didn't save the optimizer, KeyError

`best.pt` saved only `{model, it, eval}`, no `opt`. The resume code hard-read `ck["opt"]` → **KeyError, instant crash**.

Fix: tolerant loading (`if "opt" in ck`); and make subsequent `best.pt` save the full state `{model, opt, it, best_eval, hist}`.

Aftershock: even with tolerance, 676's `opt` was permanently lost → resuming from `best.pt` starts Adam with zero momentum → the first few warmup steps take oversized steps → nearly

nudges the good policy off. **A checkpoint must save the full state (incl. opt), or you cannot truly resume seamlessly.**

Trap 3: eval across step-horizons, best.pt records go haywire

Doing curriculum learning (shrinking the episode horizon from 2000), at one point I let eval also use the curriculum horizon. Consequence: **eval at 2000 steps is naturally higher than at 1000** (more tiles can be eaten) → a spurious "900 at 2000 steps" overwrote "702 at 1000 steps" → best.pt's recorded score was incomparable across horizons, completely haywire.

Fix: train with the curriculum horizon, but always evaluate at the 1000-step standard. That keeps eval comparable and best.pt consistent.

Trap 4: the minibatch KL guard was "fake"

To prevent KL explosions I added a "minibatch-level KL guard." But review revealed it **did nothing**:

1. the KL check was placed after `opt.step()` — that step's update already took effect, can't stop it;
2. the check used the old ratio computed before the step, so it can't measure "the KL caused by this update."

Correct implementation (compute the real KL after the step + roll back the minibatch if over threshold):

```
old_params = [p.detach().clone() for p in model.parameters()] # save params before step
opt.step()
with torch.no_grad(): # re-forward after step
    new_logp2, _, _ = model.ev(obs[b], acts[b])
    logr2 = new_logp2 - old_logp[b]
    kl_real = (logr2.exp() - 1 - logr2).mean().item() # real KL (k3 estimator)
if kl_real > THRESH * target_kl:
    for p, old in zip(model.parameters(), old_params):
        p.copy_(old) # roll back this step
    early_stop = True; break
```

Lesson 6: "looks right but is logically wrong" code is the worst kind in RL. Training takes hours; a `resume=False` or a "fake KL guard" wastes a whole night without your knowing why. Build the habit: review the key logic first (loading / resume / paths / guard mechanisms), then run one or two steps to confirm loading is correct (is the opening eval 676?), before launching a long run.

Three ways the threshold collapsed (the difficulty of tuning the KL guard)

Even with the KL guard implemented correctly, the threshold (multiple of target_kl) is extremely hard to tune:

Threshold	Result
4× (0.12)	on resume, Adam's first-step bias-correction takes a big step → every minibatch exceeds the threshold and is rolled back → total freeze, policy stuck (ev/kl/vL all 0)
20× (0.6)	too loose, a "small update but bad direction" of kl=0.08 gets through → one step nudges 702 into "grass-edge crawling"
8× (0.24)	with a large batch (2048), a single update pushes harder → eval variance blows up to ±431, it63 dies at -193

Key realization: the KL guard prevents "drifting too far (large KL)," but **can't prevent "small steps (kl=0.08) sliding toward a bad policy"** — because the kl isn't large, it doesn't trigger. The update that nudged 702 into "grass-edge crawling" had kl of only 0.08. This shows the KL trust-region alone cannot cure reward-hacking-style slow drift.

6.5 More lethal traps: no chapter of their own, but each wastes a night

Beyond the big traps, the late debugging phase hit a string of "resume / engineering" failures. Individually unremarkable, but each wasted training runs — and they interlock.

Trap 5: warmup freeze — resume from best.pt + fresh opt + strict threshold = total freeze

Resuming from 676's best.pt, the first few steps (it33–35) showed a bizarre "total freeze": **ev / kl / clip / vL all 0, eval frozen at 676.7.**

Root cause: best.pt didn't save opt → fresh Adam with zero momentum → the first step's **bias-correction makes the effective step size large** → every minibatch exceeds the 4× threshold → all rolled back by the KL guard → not a single successful update in the whole round → the policy doesn't budge.

The bizarre part: lowering lr (1e-4→3e-5) doesn't help — because Adam's bias-correction is independent of lr, the first step is big anyway.

Trap 6: resume's first step nudges the good policy off

Switching to a looser threshold (20×) stopped the freeze, but **resume's first step (it44) had kl of only 0.08, and nudged 702 into "grass-edge crawling," eval straight to 53.**

Root cause: 702 is a "sharp peak" — the surrounding policy space is steep, so a not-large update ($kl=0.08$) slides off. And this update's direction pointed toward the "grass-edge crawling" hacking attractor. **The KL guard can't prevent this "small-step slide into a bad policy."**

Trap 7: changing 4 variables at once, can't attribute

One version (v9) changed **4 things at once**: training early-stop + clip $0.2 \rightarrow 0.1$ + minibatch KL guard + resume from best.pt (fresh opt). It collapsed, but **attribution was impossible** — four variables at once, one of which (early-stop) was even the wrong direction.

Lesson: a live counter-example of violating the one-variable rule. Only by rolling back to "baseline v8, change just one thing (the KL guard)" did it become clear.

Trap 8: eval_every=1 changed the training trajectory, couldn't reproduce the stable policy

To make the "curriculum state machine" more responsive, I changed evaluation from every 10 iters to **every iter (eval_every=1)**. Side effect: **the training trajectory changed** — previously $eval_every=10$ produced it36's "small-variance (± 47) stable policy," but after per-iter eval, variance stayed large ($\pm 150-280$) and **the stable policy never reproduced**, the curriculum trigger (variance < 50) was never met, and it oscillated to death at -93.

Root cause: eval itself doesn't change training, but changing eval_every incidentally affected other things (sampling cadence / RNG stream), so the trajectory drifted. RL is unusually sensitive to such "seemingly unrelated" changes.

Trap 9: curriculum trigger too strict, the stable policy shut out

The curriculum trigger was "eval > 550 and variance < 50 ." But it62 had eval 480, variance **53** — **missing the trigger by a hair** ($480 < 550$ and $53 > 50$), so the curriculum never started, and it collapsed the next step. The stable policy was shut out by an overly strict gate.

Trap 10: the "perturbation + discard opt" experiment's accidental discovery

Trying to escape the "resume always collapses" bind, I tried "load 702's weights + discard its opt (fresh opt as perturbation)." The gif still showed "grass-edge crawling" — **but the key discovery: it44 barely updated ($kl \approx 0$), yet the gif already crawled the grass edge**, meaning **grass-edge crawling is 702's own intrinsic behavior on certain tracks (seed[0]), not a gremlin produced by a training collapse**.

Significance: eval 702 (a 5-track average) is propped up by the other tracks; on seed[0] it

crawls the grass to begin with. **Fine-tuning (opt/lr/kl) can't fix an intrinsic weakness in the policy — it needs substantive retraining.**

Unfixed debt: truncated leaking into dones' bootstrap bias

A bug present from the earliest version and never fixed: CarRacing's TimeLimit is 1000 steps, and every rollout inevitably hits 1–2 truncations. **Treating truncated as terminated introduces a bias into GAE's bootstrap** (at a time-limit truncation there is still future value; it shouldn't be bootstrapped as "episode over, no future"). This bias sat in the background throughout — known but not yet handled technical debt.

The shared lesson of this string of traps: a large share of late-stage RL debugging is not "algorithm" but engineering details like resume / checkpoint / RNG streams / trigger conditions. They **look inconsequential yet can quietly derail or waste a training run**. And they interlock: opt not saved → warmup big step → strict threshold → freeze; or loosen the threshold → slide into hacking. RL stability is layered on top of these details — none can be taken lightly.

7. The 2000-step gremlin: how a loose horizon feeds reward hacking

After conquering the S-curve and reaching 702, a new goal emerged: eval 676/702 was often "score banked before flying off mid-track," not truly completing a lap. To make it complete a lap, I raised the episode horizon from 1000 to **2000**, to give it more time.

Prediction (later borne out): I had a hunch that "2000 steps will have gremlins." Reasoning: a loose horizon → low pressure to go fast → the agent can "grind slowly and still bank score," with no incentive to be quick.

Gremlins, frame by frame

Resuming from the perfect-line iter60 (99% mid-track at 1000 steps) into 2000-step training, the behavior evolved:

iter	on-track %	speed	behavior
61	52%	—	runs half, flies into grass, stuck (318 frames on grass)
62	38%	—	flies off even earlier, eval 714 is score banked before
63	75%	2.4	learns to escape grass: gets out after 106 frames
64	99%	1.8	perfect line: full mid-track, only enters grass at the start

65	99%	1.8	stable in "slow-turtle," 2000 steps can't finish a lap
66	—	—	slides into "grass-edge crawling," eval 517 → 111

The gremlin's two forms:

1. **slow-turtle** (it64/65): to avoid the grass, it slowed to 1.8 — 2000 steps can't even finish a lap, eval only 490–517. A loose horizon let it "grind slowly."
2. **grass-edge crawling** (it66): from "inside-line edge, still on track" it slid into "pressing the grass, crawling the edge," eval collapsed to 111. This is reward hacking — slowly scraping tiles along the boundary, a "good enough but low-efficiency" attractor.

Changing the config (n_steps 40000 + mb 2048, hoping big data + big batch would stabilize) didn't help, and collapsed faster: the big batch pushes harder per update, **it62 variance blew to ± 431 , it63 died at -193.**

Lesson 7: the episode horizon is an overlooked, powerful hidden hyperparameter. 2000 steps (loose) is a gremlin nursery — the return scale grows (easy to collapse), the pressure to go fast disappears (laziness), and it gives room for "slow edge-scraping" (reward hacking). By contrast 1000 steps presses harder, with high pressure to be fast. A loose constraint breeds laziness and shortcuts — a counter-intuitive but deep rule in RL.

7.5 A "zoo of attractors" in training

Throughout debugging, the agent fell into all sorts of **attractors** — once trapped, it stably repeats some bizarre behavior and is extremely hard to escape. These are RL's most vivid failure modes, worth their own archive. Fundamentally they are a superposition of **local optima** (parameter space) and **absorbing states** (state space), in three classes.

Class 1: exploration-failure type (trapped, randomness can't escape)

① Spinning in place on grass (after flying off track)

The car flies off into grass, circles in place, spiral tracks behind it. **The more it spins the further it drifts from track**, so it gets ever harder to return — an "ever-deeper" absorbing state.

Why random exploration can't escape: returning to track requires "driving a stretch in a specific direction" (a low-probability precise event); grass is a large area, random actions mostly stay in grass → can't hit "return to track." No std helps (large std just spins randomly, still missing).

② Spinning at the start / in place (after one $kl=0.68$ violent update at it35)

Spins in place from the start, steering hard-locked without recentering. A single overly violent

update (kl=0.68) pushed the policy to the "hard-lock" extreme → the car keeps turning the same way → spins in place.

Class 2: reward-hacking type (good-enough but a shortcut, fed by positive feedback)

③ Crawling the road/grass edge (it66)

The car crawls the road/grass boundary, half on the track edge, half on grass, slowly scraping tiles. On-track % looks okay, but net score is low (time penalty > tiles scraped).

Why it's stubborn: crawling the edge earns a little positive feedback (touching boundary tiles), and the KL guard can't stop this "small-step (kl=0.08) slow slide into edge-crawling" — it's not an overshoot, it's a slow drift.

④ Flickering to scrape tiles (after the it35 collapse, road/grass switched 18 times)

Unlike edge-crawling: this is **high-frequency in-and-out** — scrape a stretch of track (RRRR) → step out (...) → scrape again → a regular cycle. A uniform flicker pattern, stably inching forward for score, but half the time on grass, low-efficiency.

Class 3: over-conservative type (lazy/playing dead, forced out by constraints)

⑤ Playing dead, not moving (the off-track penalty -1 run)

Learned to stay still — an off-track -1 is too heavy, "not moving = not off-track = safest." std frozen at 1.0, the policy locked into this absorbing state. eval -1045.

⑥ Parking on track / slow-turtle (2000 steps, it64/65)

Perfect line (99% mid-track) but slow at 1.8 — 2000 steps can't finish a lap. A loose horizon lets it "grind slowly and still bank score," with no incentive to speed up. A laziness attractor fed by a "loose horizon."

Class 4: control-instability type (oscillation from unlearned recentering)

⑦ Hard-lock and circle (it30)

On exiting the S-curve, steering fixes at some nonzero angle without recentering → the car stably circles. Recentering "didn't come back enough."

⑧ Reverse spin (it29)

On exiting the S-curve, recentering "overshoots" → over-corrects → spins the other way. ⑦⑧ are two directions of the same problem (exit-recentering not controlled).

The unified lesson of the attractors:

- **exploration-failure type** (spinning) — once in the absorbing state, more exploration (std / uniform dist / switching to Beta) can hardly escape, because "return to track" is a low-probability precise event.
- **reward-hacking type** (edge-crawl/flicker) — fed by faint positive feedback, the KL guard can't stop the slow drift; the cure is a training cost (penalize edge) + eval on the true reward.
- **over-conservative type** (playing dead/slow-turtle) — fed by too-heavy penalties or too-loose horizons; the fix is to change the constraint itself.
- **control-instability type** (circling/reversing) — precise control (recentering) not learned; small std + more training.

The shared solution philosophy: prevention > escape. Once in an attractor, escaping is extremely hard; better to use precise control (small std, log_std_init) to never fall in. This is a lesson CarRacing taught me over and over.

8. Two late insights: rethinking sampling and reward

By this point in debugging, the traps had risen from "algorithm parameters" to "training-mechanism design." Two insights surfaced, each corresponding to a legitimate technical direction in RL.

Insight one: smart sampling — don't let attractor samples dominate training

A rollout of 20000 steps, ~20 episodes; if **most samples are trapped in the same attractor** (grass spinning, edge-crawling, standing still), training gets dominated by these "dead-loop samples" → learns crooked. The envisioned improvement:

- give higher **sample weight** to samples that **escape the attractor on their own** (recovery, breakthrough);
- allow dirty samples (noise/failure samples carry "don't do this" info), but **noise must not become the mainstream**;
- let a variety of noisy samples enter the main sampling → increase robustness, possibly **without hard-limiting std around 0.35**.

The corresponding legitimate methods: this is essentially the idea of **Prioritized Experience Replay + sample diversity/de-biasing** — only with the priority criterion changed to "does it escape the attractor." The core: an exploration-exploitation balance at the sampling level may be more fundamental than hard-limiting std.

Insight two: training cost $\neq$ eval reward (we conflated them before)

Throughout this debugging I resisted "changing the reward," because a change makes the eval score change too, making scores incomparable. But the crux is — **the reward used for training and the metric used for evaluation should be separate to begin with:**

- **Training:** may use a **shaped reward / cost function** (e.g. "car near the grass edge $\rightarrow$ reward drops") to guide learning (don't crawl the edge);
- **Evaluation:** always use the **system's true score** (raw reward), measuring true performance.

This resolves a long-standing worry: I dared not change reward before, because "training and eval used the same reward, one change and it's chaos." The right way: **the grass penalty is used only in training (a cost to guide against edge-crawling), and eval always uses the raw reward (true, comparable).** This both cures reward hacking (edge-crawl/flicker) via the cost, and keeps eval objective. This is exactly the **correct use of reward shaping:** training may "cheat" to guide, evaluation must be honest. Conflating training cost with eval reward before was a conceptual error.

9. Methodology distilled

What a 34-step debugging march distilled is not a hyperparameter, but a transferable methodology:

- **Read numbers + gif, trust the gif on conflict.** eval 452 was spin-banked, 714 was banked-before-flying-off, the 99%-on-track "slow-turtle" — these truths hide in behavior, numbers can't see them. Further: frame-by-frame quantification (on-track %, road/grass switch count, the point of flying off, speed-by-segment) beats the naked eye.
- **One variable at a time.** Only then can you attribute. One version changed 4 things at once (early-stop + clip + KL guard + fresh opt), and the result was so dirty attribution was impossible — a lesson from violating my own rule.
- **Checkpoint every step.** Models are small (19MB), 5TB of space is plenty. It was exactly "save iter{it}.pt each round" that rescued iter60/iter64 — "not recorded by best.pt but perfect-line" states — which later became reusable upstream initial models.
- **Save the full checkpoint state.** model + opt + it + best_eval + hist, none missing, or you can't truly resume seamlessly.
- **Review before running.** Training takes hours; traps like resume=False / fake KL guard / eval-across-horizons — five minutes of review saves a whole night.
- **Prevention > escape.** Once in an absorbing state (grass spinning), escaping via exploration is extremely hard; better to use precise control (small std) to prevent falling in from the start.

Coordinates of this small milestone: starting from a single environment, a Gaussian policy, and hand-written PPO, reached eval 702 and trained iter60/64 — a "99% full mid-track"

perfect-line policy — mid-to-upper on the HuggingFace CarRacing leaderboard (many PPOs stall at 400–500; 800+ needs LSTM/Beta + 8 parallel envs + 4M steps). More importantly: **having walked through reward misspec, KL explosion, critic decoupling, the std starting point, reward hacking, and four checkpoint traps — and grown from them the ability to "understand the RL learning process."**

10. Final result: the full path from collapse to iter60's "perfect line"

After walking through every trap, the best policy was not some transient high-score peak, but **iter60 — a stable policy that drives 99% full mid-track with strong generalization**. How was it "salvaged" out of a sea of collapses and gremlins? Here is the full path.

The chain of key turning points

Step	Action	Effect
1	remove the off-track penalty, back to raw reward	the policy has a direction to learn, std starts dropping
2	log_std_init = -1 (std ≈ 0.37)	precise actions, eval variance $\pm 138 \rightarrow \pm 2-4$, no more wobble
3	epochs=1 / mb=1024 / n_steps=20000	suppress KL explosion, ep_ret variance plummets
4	conquer the S-curve (grazing $\rightarrow$ recentering)	eval jumps to 676$\rightarrow$702
5	resume from a well-lined intermediate state (not the collapse point)	learns grass escape (75% on-track)
6	keep training to iter60/64	99% on-track, full mid-track, grass only at the start

The key engineering decisions that "salvaged" iter60

Per-step checkpointing saved it. iter60's eval (1000 steps, 5 fixed tracks) was only 285 — because it is **slow-but-stable** (perfect line but low speed, can't finish a lap in 1000 steps), so best.pt (which saves the highest eval) never recorded it. It was the decision to "save iter{it}.pt each round" that preserved this "low-score but perfect-line" precious policy. Later verified by frame-by-frame quantification: iter60 has **99% on-track, road/grass switch of only 1, full inside-line (shortest path)** — an ideal "upstream initial model." And evaluated at 2000 steps, its true level is as high as **eval 630** (a slow policy given enough steps can finish more).

Why iter60 is "better" than the eval-702 one:

- 702 has variance ± 200 , and on some tracks (seed[0]) it actually **crawls the grass** — the high score is propped up by other tracks;
- iter60 has tiny variance, **consistently full mid-track on every track** — the line is "right," just slow;
- **correct line (hard) + slow (easy to fix) > fast (already have it) + wrong line (hard to change)**. iter60 is short only on "speed," a fixable gap — a better downstream starting point.

Result

- **Best policy:** iter60, 99% full mid-track, eval 630 at 2000 steps, ep_ret 448 (20 random tracks, strong generalization).
- **Positioning:** single environment + Gaussian policy + hand-written PPO, mid-to-upper on the HuggingFace CarRacing leaderboard (many PPOs stall at 400–500; 800+ needs LSTM/Beta + 8 parallel envs + 4M steps).
- **Next steps (downstream exploration, all warm-started from iter60):** curriculum speed-up, multi-model weighted ensembling (speed complementarity), CNN-feature reuse (e.g. + a GBDT action head), architecture change (LSTM/Beta), and the smart-sampling & training-cost/eval-reward separation proposed at the end of this piece.

Artifacts

All training artifacts land in Google Drive: [MyDrive/carracing/<RUN>/](#) (gifs / ckpt / metrics.csv), with checkpoint-resume and per-step archiving.

- **Eval gifs of the best policy** (99% full mid-track):
drive.google.com/drive/folders/1Vy7AWiFSUMyI7_GwcNyKlGKuMeueN6dS
- **Best model checkpoint** (iter60):
drive.google.com/file/d/1dwzW1EzzCK5QOY11JADtMjQ8pAILBc59

A closing line: the most valuable thing on this road was not reaching eval 702, but training iter60 — a "perfect-line" policy — and **understanding how it was salvaged, step by step, out of a sea of collapses, gremlins, and attractors, via "raw reward + small std + per-step archiving + reading behavior frame-by-frame."** Scores go stale, but this ability — "locating a good policy amid chaos" — is transferable, real skill.

— End of Part 4. Next stop: landing experiments for smart sampling & training-cost/eval-reward separation. —